BIOINFORMATICS

Workshop 1: Global and Local Alignment in Python

Prof. Magnus Bordewich

Learning Objectives

- Translate the Needleman-Wunsch recurrence from the lecture directly into working Python code.
- Confirm, by running your code, that both reproduce the worked examples from the lecture slides.
- Compare Human beta-globin DNA encoding with that of 12 other vertibrates.
- Extend it into a Smith-Waterman local-alignment implementation.
- Incorporate Blossum matrix scoring for amino acids.
- Implement affine gap penalties.

Background

Lecture 1 derived the global-alignment recurrence V[i,j] = max(diagonal, up, left), with base cases V[i,0] and V[0,j] built from repeated indel penalties, and showed how to trace an optimal alignment back out of the completed matrix. Local alignment (Smith-Waterman) adds a fourth 'start afresh here' option scoring 0, resets the first row/column to 0, and starts traceback at the matrix's largest value, stopping at a 0.

The starter code align.py is a direct implementation of the global-alignment algorithm: a scoring function sigma(a,b), an align(s,t) function that fills V while recording which case ('up', 'diag' or 'left') achieved the maximum at each cell, and a traceback that follows those recorded choices back to V[0,0].

Coding Activity

1. Open the starter code align.py and run it. Re-run align() on s1="GCGCATGGATTGAGCGA" s2="TGCGCCATTGATGACCA" and confirm the optimal score is +3, matching the lecture's worked example. Read through the code and check you understand each line.
2. Adjust the sigma function to match the example with indel cost -1. Run the code on s1 = 'gttacactgtaccgtctgcact', s2 = 'cactcgggtgtacgtctgcact' and compare with the lecture results.
3. Add a function print_matrix(s, t) that fills and prints the full V matrix, formatted as a grid with row/column labels, so you can visually check it against the lecture slide. Highlight the traceback path.
4. Process the sequences for DNA encoding of beta-type globin proteins in thirteen vertebrates and compute pairwise alignment scores. File 02_globin_cds.fasta

5. Run the alignment on seq1 ='TTCCAATCTCGAGATCGGATC', seq2 ='GATCGGATCTTGGCCTAGCTA'. Observe that both sequences contain the substring “GATCGGATC”
6. Write a new function local_align(s, t) implementing Smith-Waterman: add the 0-option to the per-cell maximum, reset the first row/column to 0, find the cell with the largest value in the whole matrix, and trace back from there until a 0 is reached. Test it on seq1 ='TTCCAATCTCGAGATCGGATC', seq2 ='GATCGGATCTTGGCCTAGCTA'.
7. Compare pairwise the global and local alignments of the beta-globin mRNA transcripts for horse, rabbit and chicken in local_example.fasta. What do you notice?
8. Time your implementation (Python's time.perf_counter) on randomly generated DNA sequences of length 100, 500, 1000 and 2000. Check the runtime grows roughly quadratically, as the O(mn) analysis predicts.

Extension Tasks (Optional)

a) Use the code in blosum.py to generalise sigma(a,b) to use the BLOSUM scoring matrix for amino acids when appropriate, and compare the amino acid sequence alignments corresponding to the DNA sequences above - what do you notice? File 01_globin_protein.fasta

b) Adjust your code so that the first indel in a contiguous subsequence in the alignment scores -11, but subsequent indels that extend that gap only score -1.

c) Implement the linear-space score-only computation from the lecture (fill V column by column, keeping only two columns in memory) and confirm it returns the same optimal score as align(), using much less memory for long sequences.

Looking Ahead

Keep align.py and local_align() to hand — Workshop 2 builds heuristic search machinery on top of exactly this scoring framework.